If the parameter is really not used, you can also leave off the name of the parameter and just write an asterisk:

```
class Child < Parent
  def do_something(*)
    # our processing
    super
  end
end
```

And, starting in Ruby 3.2, you can pass the anonymous splat parameter to another method without giving it a name.

```
class Example
  def method_1(*)
    method_2(*)
  end

  def method_2(*array_args)
    puts array_args.join(", ")
  end
end

puts Example.new.method_1("a", "b", "c")
```

produces:

```
a, b, c
```

You do have to give the splat a name if you want to use it, though, so you can't write *.join or something like that.

You can put the splat parameter anywhere in a method's parameter list, allowing you to write this:

```
def split_apart(first, *splat, last)
  puts "First: #{first.inspect}, splat: #{splat.inspect}, " +
    "last: #{last.inspect}"
end

split_apart(1,2)
split_apart(1,2,3)
split_apart(1,2,3,4)
```

produces:

```
First: 1, splat: [], last: 2
First: 1, splat: [2], last: 3
First: 1, splat: [2, 3], last: 4
```

In practice, we find this confusing, but your milage may vary.

If you cared only about the first and last parameters, you could define this method using the bare asterisk syntax:

```
def split_apart(first, *, last)
```

You can have only one array splat parameter in a method—if you had two, parameter assignment would be ambiguous. You also can't put parameters with default values after